Task 8: State Space Model

Craig Orman

Data Creation

To begin the creation of our state-space model we define the following dataset, considering only the single location so far. We consider measurement noise to be N(0, 1). We also set the truncation level to 20

```
m = 20 # Truncation

mu <- 3
sigma_W <- 1

A <- artsim(
  n = 300,
  d = 0.3,
  lambda = 2,
  phi = 0.45,
  theta = 0.3
)

A <- A + mu + rnorm(300, sd = sigma_W)

mat = matrix(A, nrow=1, ncol=length(A),
             byrow=TRUE)
```

Variable Approximations

Then we collect our necessary values from the spatial and temporal aspects. The spatial components here are minimal on the grounds that it is a single location. Please also note, as per Fereirra, we assume that the temporal aspect is the same for all locations, so we can sample only the first one.

```
# Temporal stuff
model = artfima(mat[1,], arimaOrder = c(1,0,1))

# spatial stuff
range_eta <- 1
nu_eta <- 1.5


# Defining some variables we will need later
sigma2_eta <- model$sigmaSq  # or whatever artfima() outputs
Sigma_eta <- matrix(sigma2_eta, 1, 1)
```

Next we need to convert our values to their causal representation! We note below that there are two seperate components we need to compute. The fractional tempering section, and the ARMA component, then you just flop them together. We will go through each piece separately, then bring them together

$$
\begin{aligned}
ARFIMA: \phi(B)(1-e^{-\lambda}B)^d X_t &= \theta Z_t \\
X_t &= (1-e^{-\lambda}B)^{-d}\frac{\theta(B)}{\phi(B)}Z_t \\
X_t &= \sum_{j=0}^{\infty}\psi_j(d,\lambda)Z_{t-j}
\end{aligned}
$$

Fractional piece:

```
psi_frac <- function(m, d, lambda) {
  psi <- numeric(m + 1)
  psi[1] <- 1
  for (k in 2:(m + 1)) {
    psi[k] <- psi[k - 1] * ((k - 2 + d) / (k - 1)) * exp(-lambda)
    # the k mismapping is due to starting at 1 not 0 in the index
  }
  psi
}
```

Recall the Generalized Binomial Theorem

$$
\begin{aligned}
(x+y)^r &= \sum_{k=0}^{\infty}\binom{r}{k}x^{r-k}y^k \\
(1-e^{-\lambda}B)^{-d} &= \sum_{k=0}^{\infty}\frac{\Gamma(d+k)}{\Gamma(d)\Gamma(k+1)}e^{-\lambda k}B^k \\
\psi_k &= \frac{\Gamma(d+k)}{\Gamma(d)\Gamma(k+1)}e^{-\lambda k} \\
\frac{\psi_k}{\psi_{k-1}} &= \frac{k-1+d}{k}e^{-\lambda} \\
\psi_k &= (\psi_{k-1})\frac{k-1+d}{k}e^{-\lambda}
\end{aligned}
$$

Next up the ARMA section, which is a wrapper for the ARMAtoMA function. See Brockwell 1991 for more information

```
psi_arma <- function(m, ar = numeric(0), ma = numeric(0)) {
  # ARMAtoMA returns psi_1,...,psi_m
  psi_tail <- ARMAtoMA(ar = ar, ma = ma, lag.max = m)
  c(1, psi_tail)
}
```

And then the actual function for it is pretty simple. We smash em together with multiplication.

```r
psi_artfima <- function(m, d, lambda, ar = numeric(0), ma = numeric(0)) {

  psi_f <- psi_frac(m, d, lambda)
  psi_a <- psi_arma(m, ar, ma)

  psi <- numeric(m + 1)

  for (k in 0:m) {
    psi[k + 1] <- sum(
      psi_a[1:(k + 1)] * rev(psi_f[1:(k + 1)])
    )
  }

  psi
}

psi <- psi_artfima(
  m       = m,
  d       = model$dHat,
  lambda  = model$lambdaHat,
  ar      = model$ar,
  ma      = model$ma
)

print(psi)
```

```
##  [1]    1.000000    1.199187    1.268064    2.295247    3.505827    4.778833
##  [7]    7.076272   10.583086   15.362368   22.438847   33.022028   48.384441
## [13]   70.823309  103.845347  152.229793  223.053104  326.898452  479.128245
## [19]  702.181349 1029.079801 1508.208047
```

State Space Model Creation

Breakdown:

$G_t(s)$ is an observation matrix

$M_t(s)$ is a vector of exogenous or predetermined values

$W_t(s)$ is an observation noise matrix with variance R_W

$F(\cdot)$ is a state transition operator

$V_t(s)$ is a covariance matrix

```r
# Dimensions
n_loc <- nrow(mat)
p <- 1
state_dim = m + 1

# Observation noise
#TODO: this is a possible contribution I could add!!! Fereirra assumes this is 0 actually
sigma2_W <- sigma_W^2 # measurement error variance
R <- diag(sigma2_W, n_loc)
```

```r
# Observation equation: Y_t = G*X_t + M*beta + W_t
G <- matrix(psi[1:(m+1)], nrow = n_loc, ncol = m + 1)  # [psi_0, psi_1, ..., psi_m]
M <- matrix(1, nrow = n_loc, ncol = p) # Design matrix
beta <- matrix(model$constant, nrow = p, ncol = 1)


# State equation: X_t = F*X_{t-1} + H*eta_t
F <- matrix(0, state_dim, state_dim)
F[2:state_dim, 1:(state_dim - 1)] <- diag(state_dim - 1)

H <- matrix(0, state_dim, n_loc)
H[1:n_loc, 1:n_loc] <- diag(n_loc)
```

Example Run through

$$
\begin{aligned}
X_{1|0} &= FX_0 \\
Y_{1|0} &= GX_{1|0} + M\beta \\
\Omega_{1|0} &= F\Omega_0 F^T + H\sigma^2(\eta)H^T
\end{aligned}
$$

```r
# Initial state
X_0 <- matrix(0, nrow = m + 1, ncol = 1)  # X_0|0 = 0
Omega_0 <- diag(1000, m + 1)  # P_0|0 = large initial uncertainty

# One-step ahead state prediction: X_1|0 = F*X_0|0 + H*eta_1
# But we don't observe eta_1 yet, so for prediction:
X_1_given_0 <- F %*% X_0  # E[X_1 | Y_0] = F*X_0|0 (since E[eta_1] = 0)

# One-step ahead observation prediction: Y_1|0
Y_1_given_0 <- G %*% X_1_given_0 + M %*% beta

# Prediction variance for X_1|0:
Omega_1_given_0 <- F %*% Omega_0 %*% t(F) + H %*% sigma2_eta %*% t(H)

# Prediction variance for Y_1|0:
sigma2_Y_1_given_0 <- G %*% Omega_1_given_0 %*% t(G) + sigma2_W
```

#Update and Predict steps

Predict broken down

$$
\begin{aligned}
X_{t+1|t} &= FX_t \\
C_{HV} = cov(HV_t(s), HV_t(s')) &\text{ spatial covariance} \\
\Omega_{t+1|t} &= F\Omega_t F^T + C_{HV} \\
Y_{t+1|t} &= GX_{t+1|t} + M\beta
\end{aligned}
$$

```r
# Prediction step: t|t -> t+1|t
predict_step <- function(X_t, Omega_t, F, H, Sigma_eta, G, M, beta) {
  # One-step ahead state prediction
```

```r
  X_t1_given_t <- F %*% X_t

  # State covariance prediction
  # C^{HV} = H * Sigma_eta * H^T
  C_HV <- H %*% Sigma_eta %*% t(H)
  Omega_t1_given_t <- F %*% Omega_t %*% t(F) + C_HV

  # One-step ahead observation prediction
  Y_t1_given_t <- G %*% X_t1_given_t + M %*% beta

  return(list(
    X_pred = X_t1_given_t,
    Omega_pred = Omega_t1_given_t,
    Y_pred = Y_t1_given_t
  ))
}
```

Update Step broken down:

```r
# Update step: t+1|t -> t+1|t+1 given observation Y_{t+1}
update_step <- function(X_pred, Omega_pred, Y_obs, F, H, Sigma_eta,
                        G, M, beta, sigma2_W) {


  Y_pred <- G %*% X_pred + M %*% beta
  innovation <- Y_obs - Y_pred # Prediction accuracy

  # Innovation variance: Delta_t = Var(Y_t - Y_hat_t)
  # Delta_t = G * Omega_t * G^T + sigma2_W
  Delta_t <- G %*% Omega_pred %*% t(G) + sigma2_W #Reliability of innovation because of error
  Delta_t_inv <- solve(Delta_t)

  # Cross-covariance: Theta_t(s)^T = Cov(X_{t+1}(s), Y_t - Y_hat_t)
  # Theta_t^T = Cov(F*X_t + H*V_t, G*X_t + W_t - G*X_hat_t)
  #           = Cov(F*X_t + H*V_t, G*(X_t - X_hat_t))
  #           = F * Cov(X_t, X_t) * G^T + H * Sigma_eta * 0
  #           = F * Omega_t * G^T
  Theta_t <- F %*% Omega_pred %*% t(G) # Maybe remove F? TBD

  # Updated state: X_{t+1|t+1} = X_{t+1|t} + Theta_t * Delta_t^{-1} * innovation
  X_updated <- X_pred + Theta_t %*% Delta_t_inv %*% innovation

  # Updated covariance: Omega_{t+1|t+1}
  C_HV <- H %*% Sigma_eta %*% t(H)
  Omega_updated <- F %*% Omega_pred %*% t(F) + C_HV -
                   Theta_t %*% Delta_t_inv %*% t(Theta_t)

  # Updated observation prediction
  Y_updated <- G %*% X_updated + M %*% beta

  return(list(
    X_updated = X_updated,
    Omega_updated = Omega_updated,
```

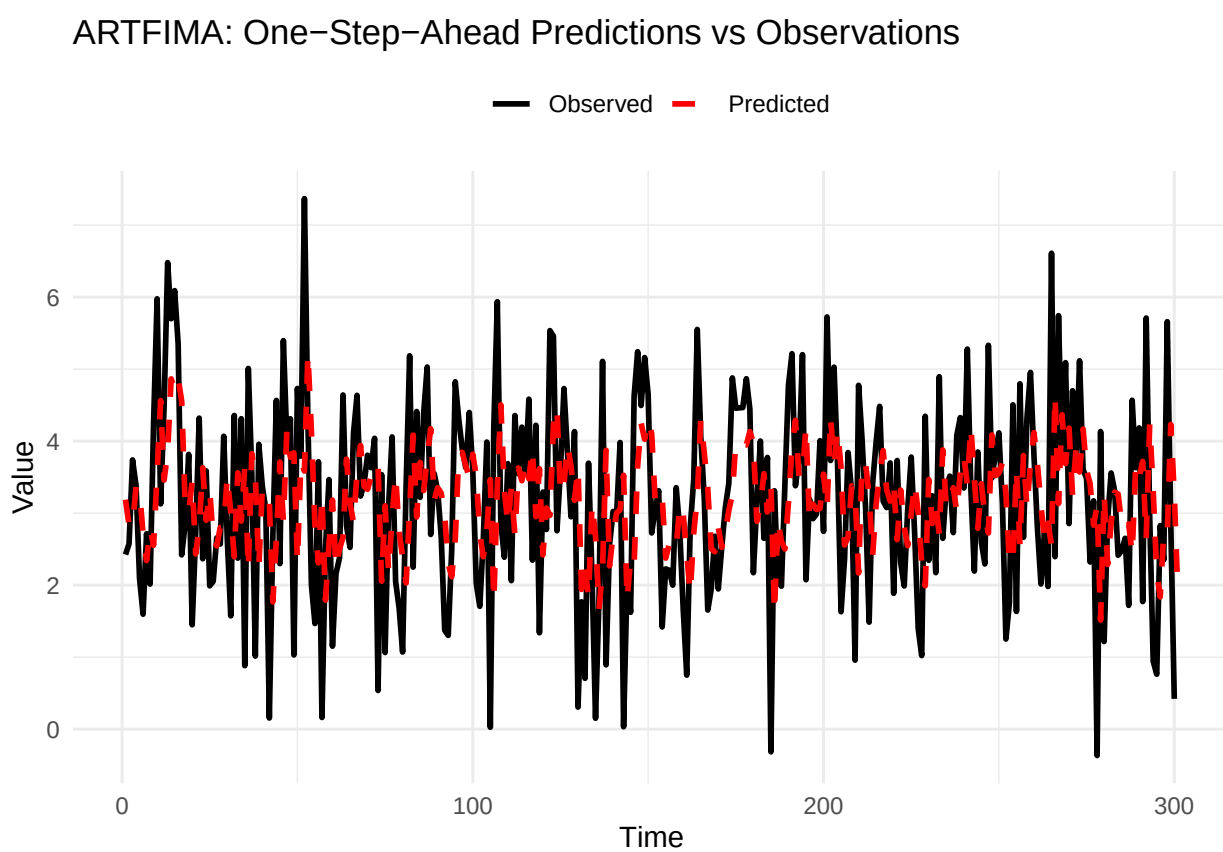
```
    Y_updated = Y_updated,
    innovation = innovation,
    Delta_t = Delta_t
  ))
}
```

Example in action

```
# Initial Vectors
Y_hat = matrix(0, nrow=nrow(mat), ncol=ncol(mat)+1)

X_t <- matrix(0, nrow = m + 1, ncol = 1)
Omega_t <- diag(sigma2_eta, m + 1)

for (i in 1:ncol(mat)) {
  # given previous instances predict.
  pred = predict_step(X_t, Omega_t, F, H, sigma2_eta, G, M, beta)
  # update values
  Y_hat[1,i] = pred$Y_pred
  X_t = pred$X_pred
  Omega_t = pred$Omega_pred

  # zzzz some time passes

  # Now you see the thing, so update our values
  Y_obs = mat[1,i]
  update = update_step(pred$X_pred, pred$Omega_pred, Y_obs,
                       F, H, sigma2_eta, G, M, beta, sigma2_W)
  # Update values
  X_t = update$X_updated
  Omega_t = update$Omega_updated
}

pred = predict_step(X_t, Omega_t, F, H, sigma2_eta, G, M, beta)
# update values
Y_hat[1,i+1] = pred$Y_pred
X_t = pred$X_pred
Omega_t = pred$Omega_pred
```

And plotting!

```
df = data.frame(predictions = t(Y_hat), originals = t(cbind(mat,NA)))
# Create time index
df$time <- 1:nrow(df)

# Reshape for ggplot
df_long <- df %>%
  pivot_longer(cols = c(predictions, originals),
               names_to = "type",
               values_to = "value")
```

```r
ggplot(df_long, aes(x = time, y = value, color = type, linetype = type)) +
  geom_line(linewidth = 1) +
  scale_color_manual(values = c("originals" = "black", "predictions" = "red"),
                     labels = c("Observed", "Predicted")) +
  scale_linetype_manual(values = c("originals" = "solid", "predictions" = "dashed"),
                        labels = c("Observed", "Predicted")) +
  labs(x = "Time", y = "Value",
       title = "ARTFIMA: One-Step-Ahead Predictions vs Observations",
       color = "", linetype = "") +
  theme_minimal() +
  theme(legend.position = "top")
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_line()`).
```